Virtual Avatar Summary Report

Date: 5th November 2026 **Timestamp:** 18:07:09 PM

Category: Technical

Learner Profile:

Name: Soumiya Ayyappan
Role: Graduate Apprentice | Domain: Graduate Apprentice
Tech Stack:

Initial Q&A Setup:

Q: Which technology or domain do you want to focus on?
A: databricks

Q: Is this for Concept Learning or a live issue?
A: Live Issue

Session Conversation:

[05:42 PM] Me:
Hello! Here is some background context about me for our session. My primary focus category for this session is technical. I am Soumiya Ayyappan, working as a Graduate Apprentice in the Graduate Apprentice domain. My tech stack includes . Here are some specific details about my project architecture: Regarding "Which technology or domain do you want to focus on?", my answer is "databricks". Regarding "Is this for Concept Learning or a live issue?", my answer is "Live Issue". Please use this information as context for my questions. Do not use a generic opening greeting. Instead, greet me as Phoebe, briefly summarize that you understand my background and focus area, and ask what we should focus on today!

[05:42 PM] PHOEBE:
Hello Soumiya, I understand you’re a Graduate Apprentice focusing on Databricks for a live issue. What specific challenge or question do you want to address today?

[05:42 PM] Me:
Can you elaborate me on that?

[05:43 PM] PHOEBE:
Certainly, Soumiya. It seems you’re working

[05:43 PM] Me:
Continue.

[05:44 PM] PHOEBE:
It seems you’re working on a retail sales and inventory optimization project, specifically focusing on the bronze layer of your Databricks architecture. In the bronze layer, raw data is ingested from your source datasets, which include sales order lines, inventory movements, and sales returns. The risk data being stored as null could indicate missing or incomplete information from the source. What specific business logic or transformation are you expecting to apply to this risk data?

[05:44 PM] Me:
and then in civil area, we are storing that in quarantine.

[05:44 PM] Me:
Everything being stored as none.

[05:45 PM] PHOEBE:
Understood. In the bronze layer, you're primarily storing raw data, including the risk data column. If this column is consistently null, it may be due to the source datasets not providing that information. Storing it as null allows you to keep the structure intact for future transformations or analysis. Shall we explore the purpose of the risk data column in your project, or would you like to discuss how to handle null values in your transformation process?

[05:45 PM] Me:
So... And also explain whether it is a batch or a stream that may be asked in my interview question.

[05:46 PM] PHOEBE:
In your case, using Auto Loader with read stream allows for efficient ingestion of data from the 10 CSV files into Delta tables. The purpose of this approach is to handle real-time data updates seamlessly while maintaining data integrity. Regarding your interview question, Auto Loader primarily supports streaming, but it can also handle batch processing, depending on how you configure it. Would you like to delve deeper into the differences between batch and stream processing in your context?

[05:46 PM] Me:
Yes, Tim.

[05:46 PM] PHOEBE:
Great! In your context, batch processing involves processing a fixed set of data at once, such as loading all 10 CSV files together. This is useful for periodic updates. On the other hand, stream processing continuously ingests data as it arrives, allowing for real-time analytics and immediate insights. What are your thoughts on which method might be more beneficial for your retail project?

[05:46 PM] Me:
Why is the rescue data stored instead of simply rejecting the entire record?

[05:47 PM] PHOEBE:
Storing the risk data as null instead of rejecting the entire record allows you to maintain the integrity

of the dataset. It ensures that you can still analyze other valid fields in the record and potentially revisit the null values later for correction or further investigation. This approach also enables better tracking of data quality issues. How do you plan to handle or address these null values in your transformations?

[05:47 PM] Me:
Where showing me when data goes into the rescue dataset.

[05:48 PM] PHOEBE:
Certainly! For your retail datasets, consider the following examples: 1. Sales Order Lines: If a sales order line is missing a product ID (which could be critical for analysis), that record might be stored with a null value in the risk data column, indicating an issue without rejecting the entire order. 2. Inventory Movements: If an inventory movement entry lacks a timestamp, it can still be stored with a null in the risk data column, allowing you to keep track of the movement while noting the missing information. 3. Sales Returns: For a sales return entry that doesn't have a corresponding order ID, the record can be stored with a null in the risk data column, highlighting the issue for future review. Would you like to discuss how to address these specific null cases in your data processing?

[05:48 PM] Me:
Yes.

[05:48 PM] PHOEBE:
To address these null cases in your data processing, consider the following approaches: 1. Data Validation Rules: Implement rules to check for critical fields like product ID or order ID. If they are null, you can flag these records for review instead of discarding them. 2. Imputation or Default Values: For fields like timestamps, consider using default values or imputation techniques based on historical data to fill in gaps where possible. 3. Quarantine Process: Move records with null values to a quarantine table where you can analyze and correct them later, ensuring they don't affect your main processing flow. How do you feel about implementing these strategies in your project?

[05:49 PM] Me:
and the quarantine records?

[05:49 PM] PHOEBE:
Yes, there is a difference: 1. Risk Data (or Rescue Data): This refers to records that contain null or invalid values in critical fields but are still retained in the dataset for future analysis. They are noted for potential issues but not removed. 2. Quarantine Records: These are records specifically set aside for further review or correction. They may contain significant errors or missing information that prevents them from being processed in the main workflow. Would you like to explore how to effectively manage these records in your data pipeline?

[05:49 PM] Me:
with the business problem statement mentioned and how we are storing it in a valid and invalid form.

[05:50 PM] PHOEBE:
In your Databricks architecture, moving from

[05:50 PM] Me:
Yes. Continue.

[05:51 PM] PHOEBE:
Moving from the bronze to the silver layer involves transforming the raw data into a more structured format that addresses specific business problems. In the silver layer, you typically apply the following transformations: 1. Data Cleaning: Remove or handle null values and duplicates. Valid records are stored in a structured format, while invalid records may be flagged or moved to a quarantine table for further review. 2. Business Logic Application: Implement calculations or aggregations that align with your business problem, such as total sales per product or inventory levels. 3. Schema Enforcement: Define a clear schema for your tables, ensuring that all necessary fields are populated correctly. This approach helps you maintain data quality and prepares the data for downstream analytics or machine learning. How confident do you feel about these transformation steps? Would you like to explore any specific transformation in detail?

[05:51 PM] Me:
over there to find out the outlier?

[05:51 PM] PHOEBE:
Outlier detection is crucial for maintaining data quality. The

[05:51 PM] Me:
Discontinue.

[05:52 PM] PHOEBE:
I understand. The Interquartile Range (IQR) method identifies outliers by calculating the range between the first (Q1) and third quartiles (Q3) and defining outliers as values that fall below Q1 - 1.5*IQR or above Q3 + 1.5*IQR. The Z-score method, on the other hand, standardizes data points based on their mean and standard deviation, identifying outliers as points with a Z-score greater than a specified threshold (commonly 3 or -3). Using these methods helps ensure that extreme values do not skew your analysis. Would you like to discuss how to implement these methods in your data pipeline?

[05:52 PM] Me:
What are the difference between all of these? Elaborate me.

[05:53 PM] PHOEBE:
Certainly! Here are the differences among various types of duplicates: 1. Exact Duplicates: These are records that are completely identical across all fields. They can occur due to data entry errors or multiple ingestions of the same dataset. Removing them is often straightforward. 2. Normal Duplicates: These may have some identical fields but differ in others. For example, two sales records might have the same product ID and date but different quantities. Identifying and resolving

these requires understanding the context of each record. 3. Business Key Duplicates: These refer to records that share the same business key (like a product ID or order ID) but may have different attributes. For example, two records for the same product could have different prices or descriptions. It's essential to determine which record is the most accurate or relevant for analysis. Understanding these differences helps in effectively cleaning your data and maintaining its integrity. Which type of duplicates do you think you might

[05:53 PM] Me:
like data validation.

[05:53 PM] Me:
checked over there in server table?

[05:54 PM] PHOEBE:
Data quality encompasses several key terms, including: 1. Data Validation: This involves checking data against defined rules or criteria to ensure accuracy and completeness. It helps identify errors before the data is used for analysis. 2. Rescue Data Handling: This refers to the process of managing records with missing or invalid values, ensuring they are flagged for review rather than discarded. 3. Data Profiling: This is the analysis of data to understand its structure, content, and quality. It helps identify anomalies or patterns that may require attention. In your silver table, these checks are essential to ensure that only high-quality, reliable data is used for reporting and analytics. How do you plan to implement these data quality checks in your workflow?

[05:54 PM] Me:
tell me over there.

[05:55 PM] PHOEBE:
In the quarantine table, you typically store records that have significant issues, such as: 1. Null Values: Records with critical fields that are null, preventing them from being processed in the main workflow. 2. Duplicates: Exact duplicates or business key duplicates that need further review to determine which record to keep. 3. Missing Values: Records with essential information missing that cannot be filled through imputation or default values. Storing these records in quarantine allows for further investigation and correction before they are either reintroduced into the main dataset or discarded. How do you envision managing these quarantined records

[05:55 PM] Me:
why are we storing that in current end records?

[05:55 PM] Me:
So kindly tell me about that.

[05:56 PM] PHOEBE:
Storing quarantine records instead of deleting them permanently is crucial for several reasons: 1. Data Integrity: Keeping these records allows you to maintain a complete history of your data, which can be valuable for audits or troubleshooting. 2. Future Analysis: Some records may contain useful

information that can be corrected or validated later, contributing to a better understanding of data quality issues. 3. Learning Opportunities: Analyzing the reasons for records being quarantined can help improve data collection processes and validation rules in the future. By working with these records later, you can enhance your overall data quality and ensure that the valid data in your silver layer is as accurate as possible. How do you plan to review

[05:56 PM] Me:
rejected, or quarantined and why.

[05:58 PM] PHOEBE:
Based on your examples: 1. Exact Duplicates: The nine exact duplicate records should be reviewed and one instance of each retained while the others are rejected to avoid redundancy. 2. Negative Quantities: The 20 records with a quantity of -999 should likely be quarantined, as negative quantities may indicate data entry errors or need further investigation. 3. High Unit Prices: The 24 records with a unit price of 999.999 should also be quarantined, as this value may be unrealistic and requires review to determine if it's a valid price or an error. 4. Null Cells: The 14 records with null values should be quarantined as well, as they may contain critical missing information that needs to be addressed. This approach ensures that your main dataset remains clean while allowing for further analysis of potentially problematic records. How do you feel about this classification process?

[05:58 PM] Me:
this.

[06:00 PM] PHOEBE:
In your architecture, the distinction between fact and dimension tables is essential for effective data modeling: 1. Fact Tables: These typically store quantitative data for analysis. In your case, the sales order lines and inventory movements would be stored in fact tables, as they contain measurable metrics like sales amounts, quantities sold, and inventory levels. 2. Dimension Tables: These provide context to the facts and typically contain descriptive attributes. For example, the product master and sales returns can be stored in dimension tables, as they provide additional details about the products and the reasons for returns. Storing data this way in the gold layer allows for efficient querying and reporting, as it aligns with the star schema design. After completing the silver table, moving to the gold layer ensures that the data is optimized for analytical queries, providing a structured and clear view of your business metrics. How confident do you feel about this distinction? Would you like to explore any specific aspect further?

[06:00 PM] Me:
Kindly elaborate me on that.

[06:02 PM] PHOEBE:
The purpose of using fact and dimension tables in a star schema is to optimize data for analytical queries and reporting. Fact tables store measurable data, allowing for aggregation and analysis, while dimension tables provide context, making it easier to filter and categorize the facts. This

structure enhances query performance and simplifies the data model. In the gold layer, ideally, there should be no null values in fact tables, as they should contain complete and accurate data for analysis. However, some null values may still exist in dimension tables, especially if certain attributes are optional or not applicable to all records. It's crucial to ensure that any null values in fact tables are addressed during the transformation process, as they can lead to inaccurate analysis. If your team is encountering null values in the gold layer, it may indicate that the data cleaning and validation processes need to be revisited. How do you plan to address these null values to ensure data quality in your gold layer?

[06:02 PM] Me:
So differentiate and tell.

[06:04 PM] PHOEBE:
Having multiple fact tables instead of a single fact table allows for more granular analysis and better organization of data. Each fact table can focus on a specific area of your business, such as sales, inventory movements, or returns, enabling targeted queries and performance optimization. In your project, if you have three fact tables (e.g., fact_sales, fact_inventory_moments, and fact_sales_returns), each would store aggregated metrics relevant to that specific aspect of the business. For example: fact_sales would contain aggregated sales data, such as total sales amount, quantity sold, and possibly metrics like average sale price. fact_inventory_moments would focus on inventory-related metrics, such as total inventory changes or stock levels. fact_sales_returns would include metrics related to returns, such as total return amounts and quantities. The whole cleaned dataset from the silver layer (e.g., silver_sales) would not be stored in the fact tables. Instead, only the relevant aggregated metrics that support analysis would be included. Dimension tables would then provide context for these facts, containing descriptive attributes like product names, categories, or time periods. This separation helps maintain clarity and efficiency in your data model. How do you feel about this structure? Would you like to explore any specific aspect further?

[06:04 PM] Me:
table with multiple dimension tables?

[06:05 PM] PHOEBE:
Deciding which columns belong in fact tables versus dimension tables typically follows these guidelines: 1. Fact Table Columns: These should contain measurable, quantitative data that can be aggregated, such as sales amounts, quantities sold, or inventory levels. Facts are often numeric and represent the performance or outcome of business processes. 2. Dimension Table Columns: These should contain descriptive attributes that provide context to the facts, such as product names,

[06:05 PM] Me:
KPIs. What is KPI? It is being also stored as stable.

[06:06 PM] PHOEBE:

A KPI, or Key Performance Indicator, is a measurable value that demonstrates how effectively a company is achieving key business objectives. KPIs are used to evaluate success at reaching targets and can be financial, operational, or strategic in nature. In a data warehousing context, KPIs can be stored in fact tables or as separate tables, depending on their structure. For example, a KPI table might include metrics like total sales, average order value, or inventory turnover rates, which can be calculated from the data in the fact tables. Storing KPIs allows for easy tracking and reporting of performance over time. How do you plan to incorporate KPIs into your project?

[06:06 PM] Me:
two to five seconds.

[06:06 PM] PHOEBE:
Joining fact and dimension tables is necessary to provide context to the quantitative data in fact tables. This allows for meaningful analysis, such as understanding sales by product category or time period. When you join a fact table with multiple dimension tables, you can create a comprehensive view that enables complex queries and reporting, combining various attributes for deeper insights. However, this may increase query complexity and processing time, especially with large datasets. How do you feel about managing these joins in your project?